

1교시: 로컬 정적 서버 실행 - 브라우저와 curl 확인

수업 목표

- 정적 파일 서버를 실행하고 HTTP로 확인한다.
- 프로세스, 포트, file 경로, HTTP 상태 코드를 하나의 확인 기록으로 연결한다.
- Day3에서는 미니앱 구현을 시작하지 않고, 최소 정적 파일만 사용한다.

오늘 반드시 가져갈 것

필수 개념	왜 필수인가	놓치면 생기는 문제	확인 기록
현재 경로	http.server는 실행한 디렉터리 의 파일을 제공한다.	다른 폴더를 열어 놓고 "파 일이 없다"고 오해한다.	pwd, ls index.html
서버 프로세 스	명령을 실행하면 터미널 하나가 서 버 프로세스로 점유된다.	같은 터미널에서 curl을 치려다 서버를 중단한다.	서버 터미널 유지, 새 터미널에 서 확인
포트 8000	브라우저와 curl이 접속할 주소의 입구다.	포트 충돌이나 다른 주소 접속을 구분하지 못한다.	http://localhost:8000, 상태 코드
브라우 저/curl/로그	같은 요청을 사용자 관점과 운영 관 점으로 나눠 본다.	화면만 보고 성공을 단정 한다.	URL, curl -I, GET / 로그

챌린저 복구 기준

- 서버 명령을 실행한 터미널은 그대로 두고, 확인 명령은 새 터미널에서 실행한다.
- connection refused가 나오면 서버가 꺼졌는지, 포트가 다른지, 시작 직후인지 순서대로 본다.
- 404가 나오면 서버는 살아 있는 것이므로 파일 이름과 URL 경로를 먼저 확인한다.

50분 흐름

Time	Activity
0-5분	Day2 저장소와 CLI 확인 기록 확인
5-15분	정적 서버와 동적 앱의 차이 설명
15-30분	index.html 생성, 서버 실행, 브라우저/curl 확인
30-40분	서버 터미널 로그와 HTTP 상태 코드 기록
40-50분	포트 충돌/경로 오류와 다음 교시 연결

0-5분 Day2 저장소와 CLI 확인 기록 확인

- 진행: Day2 저장소와 CLI 확인 기록 확인
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

상세 설명

정적 서버는 HTML, CSS, image 같은 파일을 그대로 HTTP로 제공한다. 오늘 만드는 index.html은 앱 구현이 아니라 서버가 file 경로를 읽고 HTTP response를 반환하는지 확인하기 위한 최소 자료다. 로그인, API, 데이터베이스, JavaScript 기능은 시작하지 않는다.

python3 -m http.server 8000은 현재 디렉터리를 기준으로 파일을 제공한다. 그래서 서버를 어느 경로에서 실행했는지가 중요하다. 같은 명령이라도 저장소 root에서 실행하면 root의 파일을 제공하고, 다른 디렉터리에서 실행하면 다른 파일 목록을 제공한다.

Visual 1: 로컬 서비스 확인 기록 흐름



그림을 볼 때는 "명령을 실행했다"에서 멈추지 말고 경로 -> 프로세스 -> 포트 -> HTTP 상태 코드 -> 서버 로그 순서로 확인 기록이 이어지는지 확인한다. 빈칸이 있으면 다음 사람이 같은 결과를 재현하기 어렵다.

5-15분 정적 서버와 동적 앱의 차이 설명

- 진행: 정적 서버와 동적 앱의 차이 설명
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

시작 상태

Day2 저장소 안에서 다음 파일만 만든다.

```
<!doctype html>
<title>Week 1 Lab</title>
<h1>Week 1 Local Service</h1>
```

Visual 2: 브라우저와 curl이 보는 것

확인 도구	보는 위치	남길 확인 기록
브라우저	화면에 렌더링된 HTML	URL, 화면에 보인 제목
curl -I	HTTP header와 상태 코드	200, 404 같은 상태 코드
서버 터미널	프로세스가 받은 request	GET / log excerpt
pwd	서버 command 실행 위치	저장소 기준 상대 경로

브라우저 화면은 사용자 관점 확인 기록이고, curl과 서버 로그는 운영 관점 확인 기록이다. 둘 중 하나만 기록하지 말고 같은 요청을 서로 다른 각도에서 확인한다.

15-30분 index.html 생성, 서버 실행, 브라우저/curl 확인

- 진행: index.html 생성, 서버 실행, 브라우저/curl 확인
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

명령 절차

```
pwd
printf '<!doctype html>\n<title>Week 1 Lab</title>\n<h1>Week 1 Local
Service</h1>\n' > index.html
python3 -m http.server 8000
```

서버를 실행한 터미널은 그대로 둔다. 새 터미널에서 확인한다.

```
curl -I http://localhost:8000
curl http://localhost:8000
```

서버 프로세스가 막 시작되는 순간에는 첫 curl이 connection refused로 실패할 수 있다. 그때는 서버 터미널이 종료됐는지 먼저 보고, 종료되지 않았다면 짧게 재시도한다.

```
for i in 1 2 3 4 5; do
  curl -I http://localhost:8000 && break
```

```
sleep 1
done
```

서버 종료는 실행 중인 터미널에서 Ctrl+C를 사용한다.

Body 수정 후 재확인

정상 응답을 한 번 확인한 뒤에는 index.html의 본문을 의도적으로 바꾸고, 그 변경이 실제 HTTP 응답에 반영되는지 다시 확인한다. 이 단계는 "파일을 고쳤다"와 "서비스에서 고친 결과가 보인다"를 구분하기 위한 연습이다.

서버를 계속 켜 둔 상태에서 다른 터미널 또는 에디터로 index.html을 수정한다.

```
printf '<!doctype html>\n<title>Week 1 Lab</title>\n<h1>Week 1 Local Service -
Updated</h1>\n<p>Day 3 checks path, process, port, HTTP status, and logs.
</p>\n' > index.html
curl http://localhost:8000
```

응답 본문에 Week 1 Local Service - Updated와 Day 3 checks path가 보여야 한다. 보이지 않으면 다음 순서로 확인한다.

증상	먼저 볼 것	이유
예전 문구가 보임	pwd, ls index.html	서버가 다른 디렉터리에서 실행 중일 수 있다.
브라우저만 예전 문구를 보임	curl http://localhost:8000	브라우저 cache 또는 새로고침 문제일 수 있다.
connection refused	서버 터미널	서버 프로세스가 종료됐을 수 있다.
404	요청 URL	/index.html과 / 경로를 구분해야 한다.

수정 확인 기록은 아래처럼 짧게 남긴다.

확인 항목	예시
changed file	index.html
changed body text	Week 1 Local Service - Updated
recheck command	curl http://localhost:8000
recheck result	수정된 문구가 응답에 포함됨

확인 질문

- 서버를 실행한 디렉터리는 어디인가?
- 브라우저와 curl -I은 각각 어떤 확인 기록을 제공하는가?
- Ctrl+C 후 다시 curl하면 어떤 증상이 나와야 하는가?
- 파일을 수정했다는 기록과 수정 결과가 서비스에 반영됐다는 기록은 어떻게 다른가?

30-40분 서버 터미널 로그와 HTTP 상태 코드 기록

- 진행: 서버 터미널 로그와 HTTP 상태 코드 기록
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

다음 주차 매핑

Docker에서는 이 실행이 container command가 되고, Kubernetes에서는 Pod 안 프로세스가 되며, AWS에서는 hosting 대상 산출물이 된다. Terraform은 나중에 포트와 hosting 리소스를 선언한다.

예상 결과

- `curl -I http://localhost:8000`은 200 상태 코드를 보여야 한다.
- `curl http://localhost:8000`은 Week 1 Local Service가 포함된 HTML을 출력해야 한다.
- 서버 terminal에는 GET / 요청 log가 찍힌다.
- 포트 8000이 이미 사용 중이면 address already in use류 오류가 난다.
- 서버 시작 직후 첫 요청만 실패하고 재시도에서 성공하면 startup timing으로 기록한다.

흔한 오해

오해	교정
index.html을 만들었으니 앱 구현을 시작한 것이다.	오늘 파일은 정적 서버 확인용 최소 파일이다. 기능 개발은 Day4 이후다.
브라우저가 열리면 모든 확인 기록이 충분하다.	URL, 상태 코드, 서버 로그, 실행 경로를 함께 기록한다.
8000번 포트는 항상 비어 있다.	다른 프로세스가 사용 중일 수 있다. 포트 충돌은 흔한 운영 증상이다.

40-50분 포트 충돌/경로 오류와 다음 교시 연결

- 진행: 포트 충돌/경로 오류와 다음 교시 연결
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

실습 확인 기록

확인 항목	값
command	<code>python3 -m http.server 8000</code>
경로	
포트	8000
HTTP 상태 코드	
server log excerpt	

학술 근거와 현업 DevOps insight

분산 시스템을 배우기 전에는 가장 작은 HTTP service를 정확히 관찰할 수 있어야 한다. SRE 관점에서 symptom은 사용자가 보는 결과이고, cause는 프로세스, 포트, file, 설정 중 하나일 수 있다. 정적 서버 실습은 이 구분을 낮은 비용으로 훈련한다.

평가 기준

기준	2점 확인 기록
50분 참여	시간 흐름에 맞춰 설명, 활동, 산출물 작성에 참여했다.
증거 산출	수업에서 요구한 note, command, table, blocker 중 해당 산출물을 구체적으로 남겼다.
전이 연결	오늘 개념이 Week2~5 기술 또는 자기 산출물과 어떻게 연결되는지 한 문장 이상 설명했다.

공식/학술 근거 링크

- MDN HTTP Overview, <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview> - local service 확인을 HTTP request/response 확인 기록으로 연결한다.
- RFC 9110: HTTP Semantics, <https://datatracker.ietf.org/doc/html/rfc9110> - 상태 코드와 resource 확인의 공식 기준이다.
- MIT Missing Semester, <https://missing.csail.mit.edu/> - shell command와 debugging 기록을 실무형 computing literacy로 다루는 근거다.